The 40th IEEE/ACM International Conference
on Automated Software Engineering (ASE 2025)

北京科技大学
University of Science and Technology Beijing

SWINBURNE UNIVERSITY OF TECHNOLOGY

DIFFFIX: Incrementally Fixing AST Diffs via Context and Type Information

Guofeng Zeng
d202110394@xs.ustb.edu.cn

Chang-ai Sun
casun@ustb.edu.cn

Kai Gao
kai.gao@ustb.edu.cn

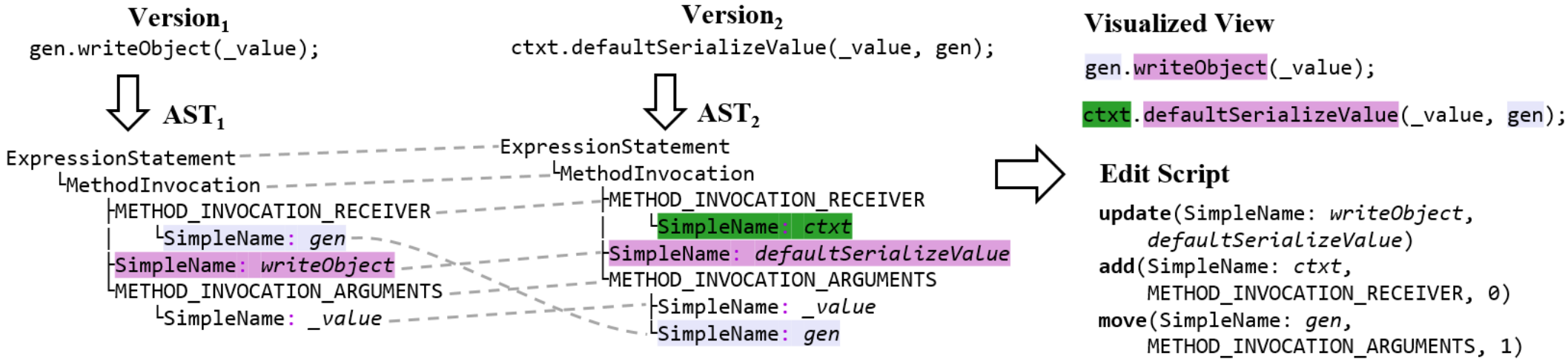
Huai Liu
hliu@swin.edu.au

Seoul, November 19, 2025

Overview of AST Differencing (ASTDiff)

- **Process**
  (1) parse two versions of programs into ASTs
  (2) compare ASTs to get the diff (node mappings)
  (3) generate edit script / visualize syntactic change

A fundamental approach to capture structural and syntactic changes of program elements



- **Techniques**
  ChangeDistiller, GumTree, MTDiff, IJM, iASTMapper, RM-ASTDiff, etc.

- **Applications**
  automated program repair, change pattern mining, refactoring detection, etc.

Accuracy Issue of ASTDiff Techniques

- **Can the generated diffs accurately reflect the code changes?**

- **Limitations of existing ASTDiff techniques**
  - Unsound node matching heuristic
  - Insufficient use of program information
  - Trade-off between accuracy and efficiency
  - Mutual interference of inaccuracies

} Struggle to generate accurate diffs, resulting in **20%-40% inaccuracies**[1,2]

Motivation:

Since inaccurate mappings are difficult to prevent, can we identify and fix them in the generated diff to improve the accuracy?

[1] Y. Fan, X. Xia, D. Lo, et al. A differential testing approach for evaluating abstract syntax tree mapping algorithms. ICSE, 2021.
[2] P. Alikhanifard and N. Tsantalis. A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools. TOSEM, 2025.

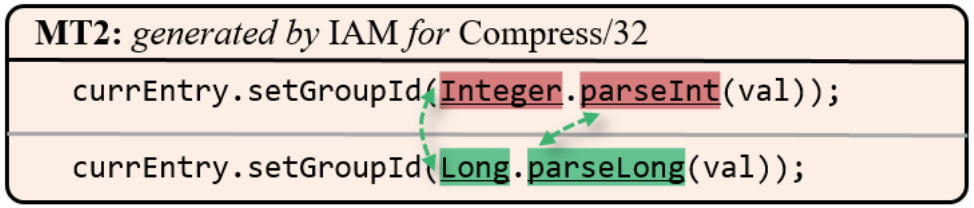
Investigation of Inaccurate Mapping Characteristics

- **Dataset: DiffBenchmark**
  - Human-verified oracle diffs for 987 commits
    - 800 bug-fixing commits from Defects4J
    - 187 refactoring commits from RefOracle

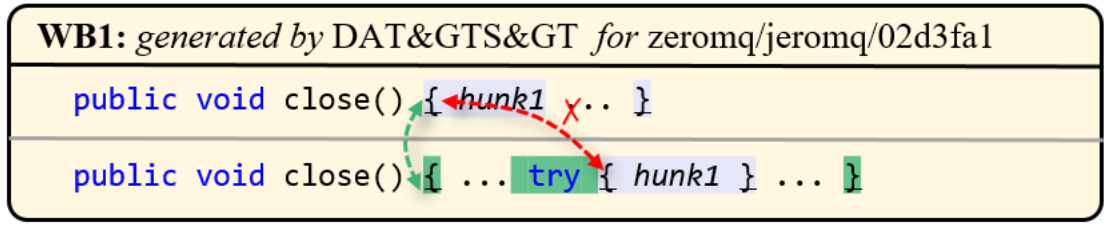
- **Subjects: 5 State-of-the-art ASTDiff techniques**
  - RM-ASTDiff (RMD)
  - iASTMapper (IAM)
  - DiffAutoTuning (DAT)
  - GumTree-simple (GTS)
  - GumTree (GT)

- **Three Inaccuracy Types**
  - Missing, Wrong, Arbitrary

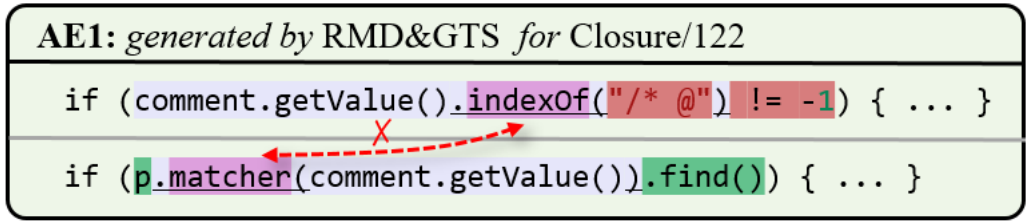
Missing Mapping



Wrong Mapping



Arbitrary Mapping



Observation #1: Context-related

■ **Inaccurate mappings commonly reside in four contexts of mapped nodes**

Parent Context

inaccuracies under mapped parents

WB_1: *generated by* DAT&GTS&GT *for* zeromq/jeromq/02d3fa1

```
public void close() { hunk1 .. }
public void close() { ... try { hunk1 } ... }
```

Children Context

inaccuracies upon mapped children

WE2: *generated by* RMD&DAT *for* Compress/46

```
if (l >= TWO_TO_32) { ... }
if (l < Integer.MIN_VALUE || l > Integer.MAX_VALUE) { ... }
```

Inner Context

inaccuracies inner mapped blocks

WE1: *generated by* RMD *for* Compress/2

```
if (offset % 2 != 0) { read(); }
if (currentEntry != null) { ... read() ... } ...
if (offset % 2 != 0) { if (read() < 0) { return null; } }
```

Nearby Context

inaccuracies around mapped statements

WS1: *generated by* RMD *for* JacksonCore/10

```
hash ^= (hash >>> 12);
hash ^= (hash << 3); hash += (hash >>> 12);
```

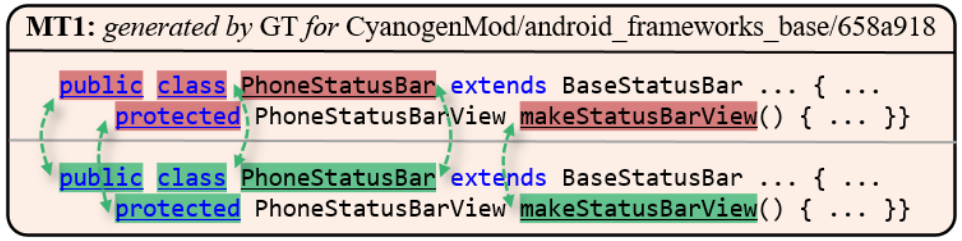
Implication:

Context can be used to locate potential inaccurate mappings.

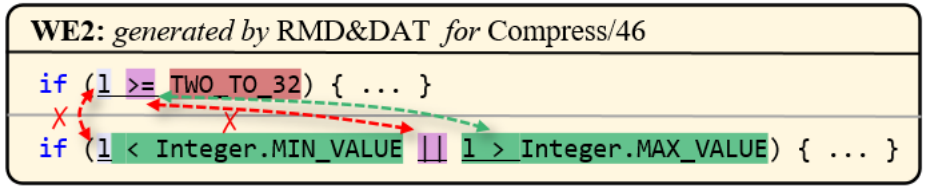
Observation #2: Type-related

- **The syntax types of nodes constraint their mappings**

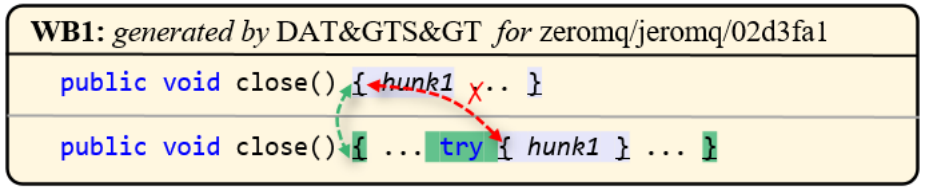
Roles in Declaration



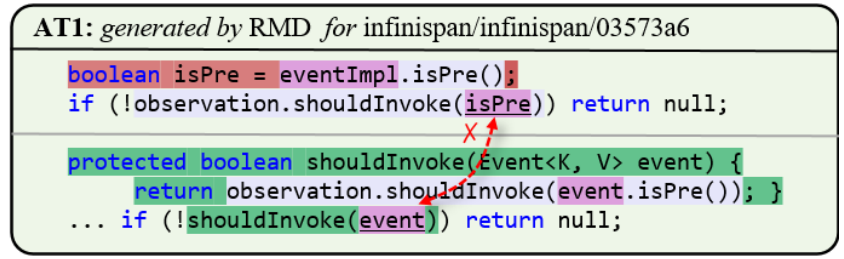
Infix Expression Compatibility



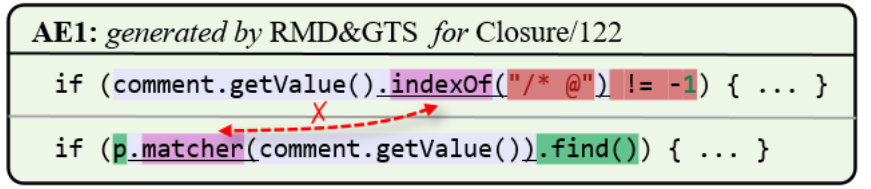
Block Ownership



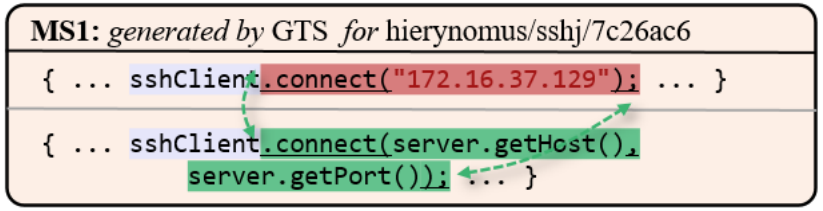
- **Certain types of nodes are prone to inaccurate mappings**



Simple Name (Variable)



Method Invocation



Statement

Implication:

Type-specific constraints should be utilized to identify and fix inaccurate mappings, especially for certain types of nodes.

Observation #3: Coexisting

- Different types of inaccurate mappings **coexist** in the generated diff, reflecting the **mutual interference** of inaccuracies

ME$_1$: *generated by* RMD *for* Cli/37

```
return token.startsWith("-") && token.length() >= 2
&& options.hasShortOption(token.substring(1, 2));
```

```
if (!token.startsWith("-") || token.length() == 1)
    { return false; }
int pos = token.indexOf("=");
String optName = pos == -1 ?
    token.substring(1) : token.substring(1, pos);
return options.hasShortOption(optName);
```

Implication:

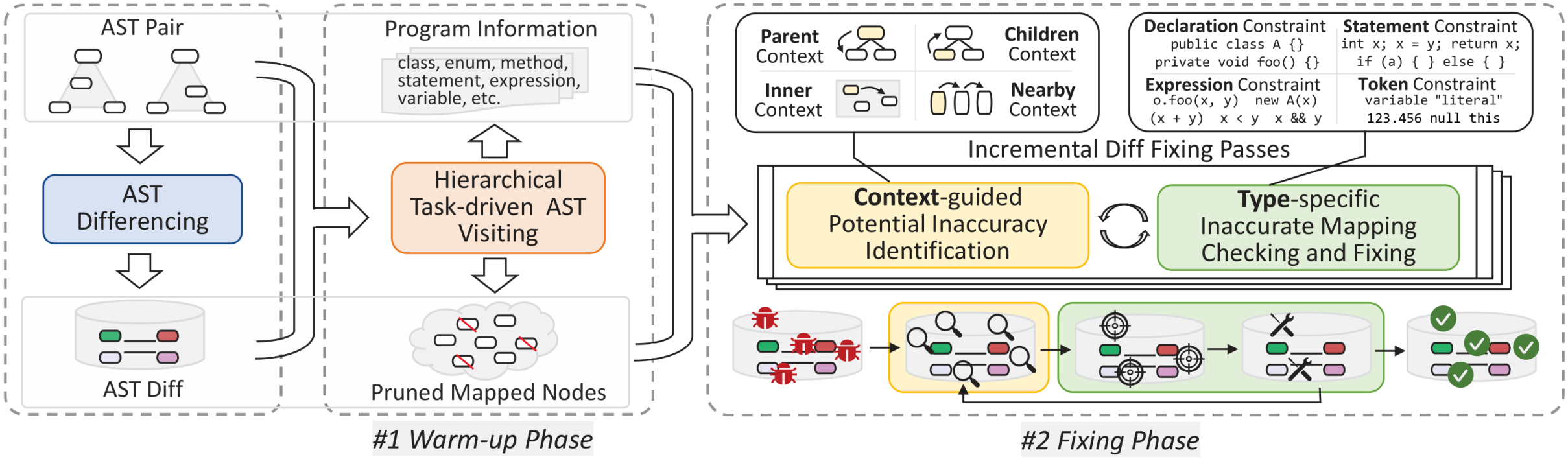
Coexisting inaccurate mappings should be fixed iteratively and incrementally.

WS&MS: *generated by* RMD *for* Time/23

```
map.put("GMT", "UTC"); ...
map.put("WET", "Europe/London");
map.put("ECT", "Europe/Paris");
map.put("ART", "Africa/Cairo");
map.put("CAT", "Africa/Harare");
map.put("EET", "Europe/Bucharest");
map.put("EAT", "Africa/Addis_Ababa");
map.put("MET", "Asia/Tehran"); ...
map.put("IST", "Asia/Calcutta");
map.put("BST", "Asia/Dhaka");
map.put("VST", "Asia/Saigon");
```

```
map.put("GMT", "UTC");
map.put("WET", "WET");
map.put("CET", "CET");
map.put("MET", "CET");
map.put("ECT", "CET");
map.put("EET", "EET"); ...
map.put("ART", "Africa/Cairo");
map.put("CAT", "Africa/Harare");
map.put("EAT", "Africa/Addis_Ababa"); ...
map.put("IST", "Asia/Kolkata");
map.put("BST", "Asia/Dhaka");
map.put("VST", "Asia/Ho_Chi_Minh");
```

Our Approach: DIFFFIX



- **Input:** A pair of ASTs and their diff generated by an ASTDiff technique
- **Output:** A refined diff with inaccurate mappings fixed

#1 Warm-up Phase performs initializations and optimizations

#2 Fixing Phase performs identification and fixing of inaccuracies

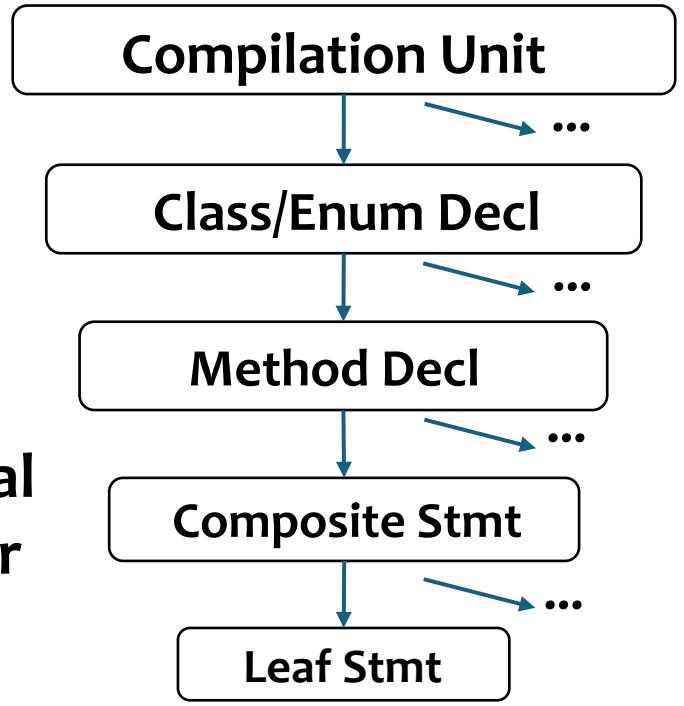
Warm-up Phase

- **Three tasks to establish the initial environment**
  - (1) ***node type checking***
    whether two mapped nodes have the same types
  - (2) ***identical mapping checking***
    whether subtrees are identical and consistently mapped
  - (3) ***global information collection***
    maintains info that across the whole AST

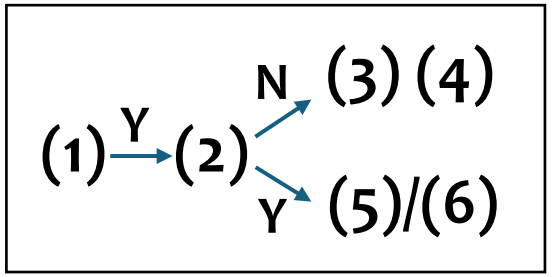
- **Three tasks to optimize the fixing phase**
  - (4) ***local information collection*** (opt_1)
    maintains shared type-specific info
  - (5) ***declaration-level pruning*** (opt_2)
    filter descendants of identical declarations
  - (6) ***statement-level pruning*** (opt_3)
    filter descendants of identical statements

(two pruned node set, one remains identical stmt roots)

Hierarchical AST Visitor

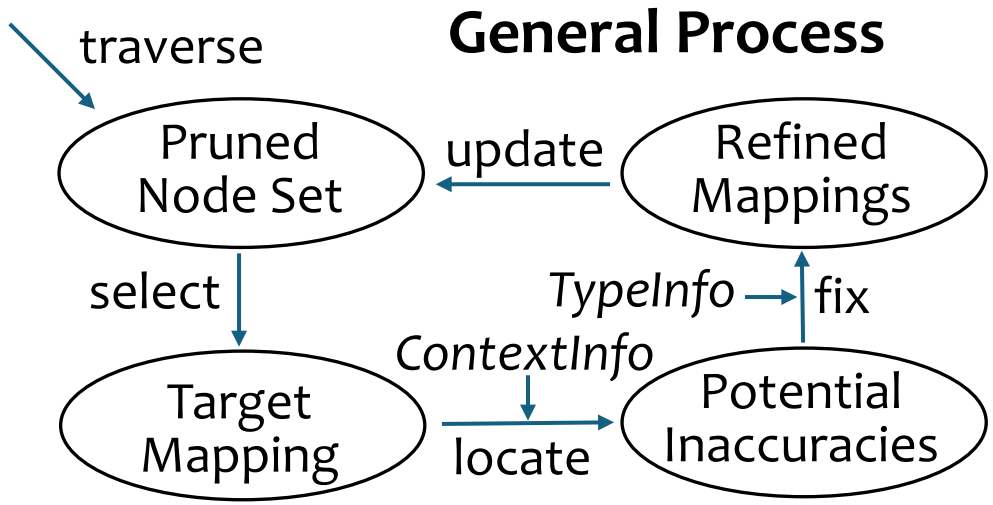


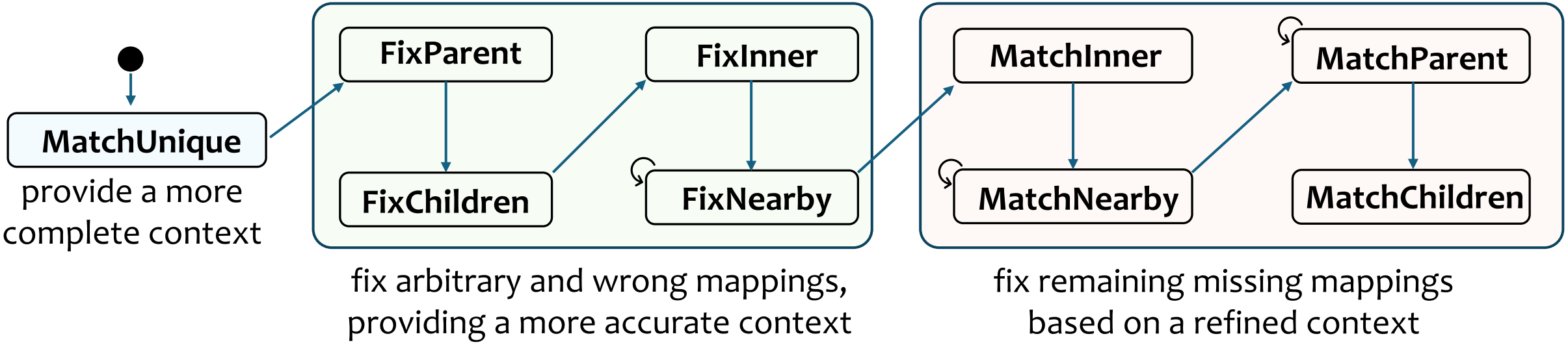
Task-driven Manner



Fixing Phase: 9 passes

- Each fixing pass traverses one of two pruned node sets, leverages a specific context to **locate potential inaccuracies around a target mapping,** and **uses both general and type-specific checks and operations** to confirm and fix them
- **The order of fixing passes** is arranged according to their **interdependencies**

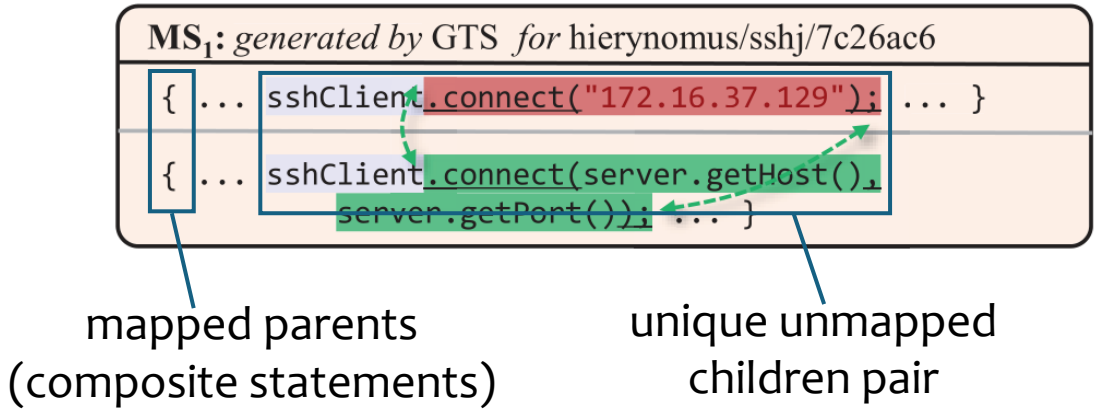




Execution Flow of Fixing Passes

Fixing Pass: MatchUnique

- **Observation:** For two mapped (a) **composite statements or** (b) **class declarations,** if each of them contains a **unique unmapped child node of a specific type,** then the pair of the two child nodes are **likely to be a missing mapping.**
- **Process**
  - **(1)** traverse the pruned node set to identify such the unique unmapped pair
  - **(2)** check whether the pair of nodes have common elements
  - **(3)** use a refined subtree matching algorithm to fix missing mappings

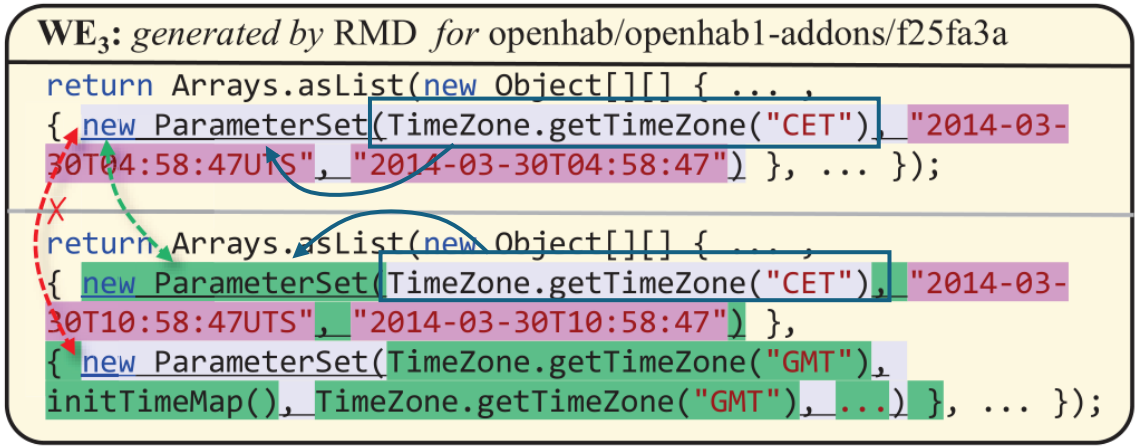


Algorithm 1: MatchUnique

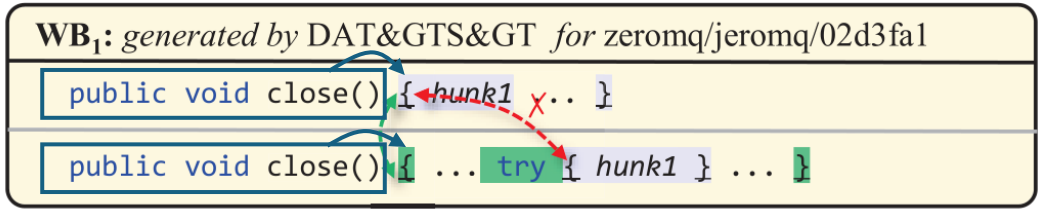
```
Data: Pruned mapped AST1 nodes P
foreach m1 ∈ P do
    if isMapped(m1) then
        m2 ← getMapped(m1)
        if isCompositeStatement(m1) then                    (a)
            U ← uniqueUnmappedChildrenPairs(m1, m2)
            foreach (u1, u2) ∈ U do
                tryToMatchInComposite(u1, u2)
            end
        end
        else if isClassDeclaration(m1) then                 (b)
            D ← uniqueUnmappedChildrenPairs(m1, m2)
            foreach (d1, d2) ∈ D do
                tryToMatchInClass(d1, d2)
            end
        end
    end
end
```

Fixing Pass: FixParent, FixChildren, and FixInner

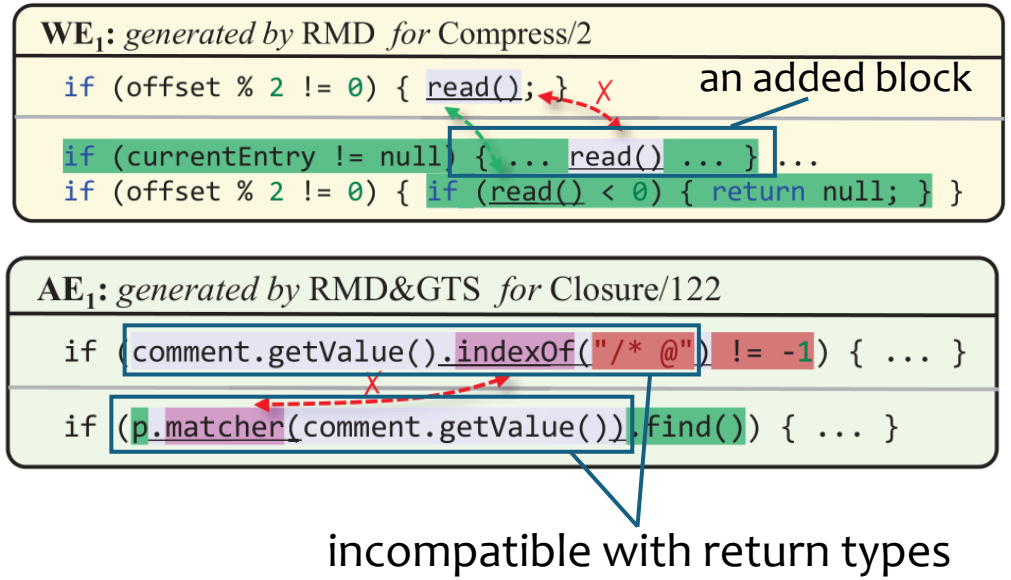
- **FixParent** identifies **suboptimal mappings of parents** that have mapped child nodes



- **FixChildren** identifies **mis-mapped child nodes** that have an unique role

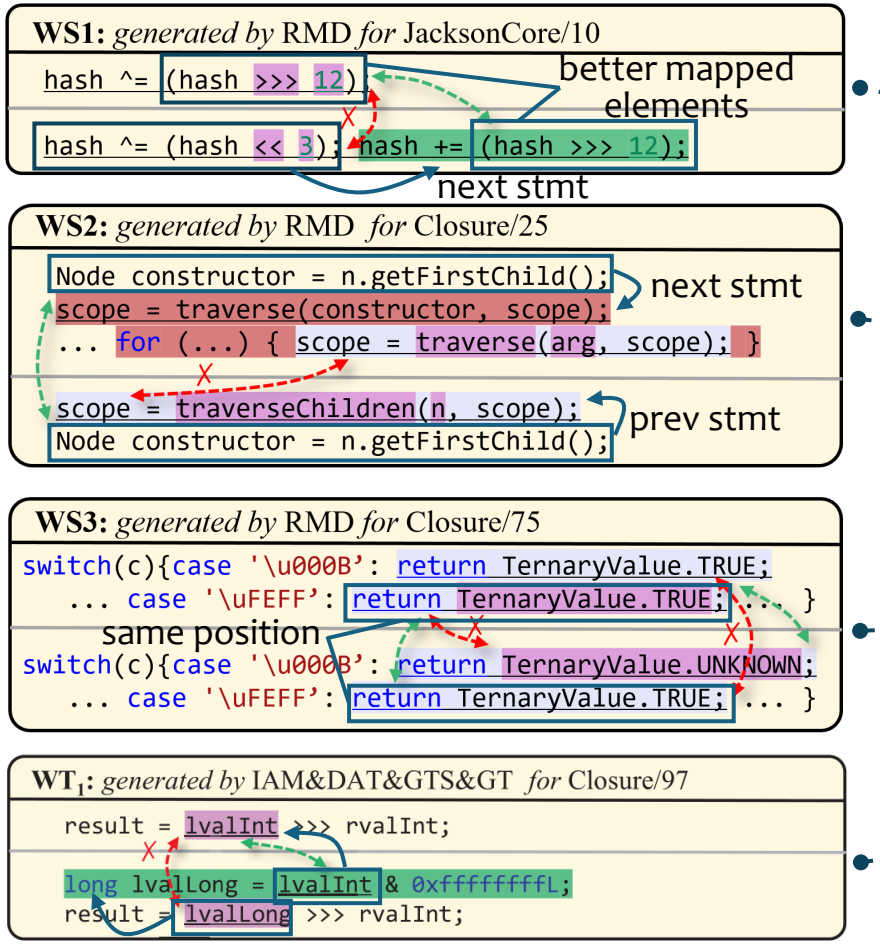


- **FixInner** identifies **arbitrary mappings**
  (1) exist solely in added/deleted **blocks**
  or
  (2) are incompatible method invocations



Fixing Pass: FixNearby

- Iteratively checks 4 types of **inaccurate mappings in nearby (previous and next) statements**
  - (1) suboptimal mappings with better nearby partners
  - (2) mis-mapped nearby statements
  - (3) mis-mapped identical statements
  - (4) mis-mapped moved nodes in nearby statements



Algorithm 2: FixNearby

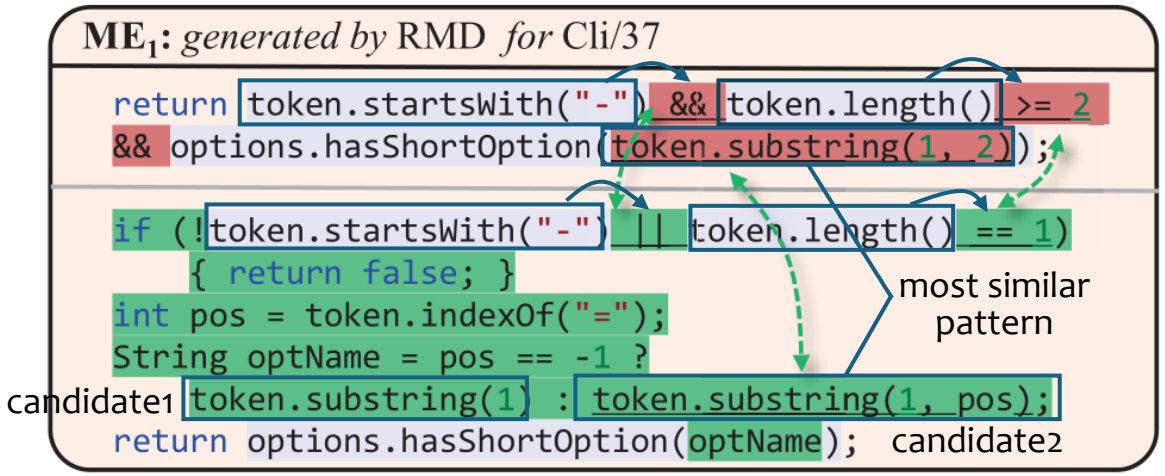
```
Data: Pruned mapped AST_1 nodes P'
1  Function FixNearbyRound(S):
2      R ← ∅
3      foreach s1 ∈ S do
4          if isMapped(s1) then
5              s2 ← getMapped(s1)
6              if checkNearbyBetter(s1, s2, R) then
7                  continue
8              end
9              checkNearbyWM(s1, s2, R)
10             checkIdenticalWM(s1, s2, R)
11             checkNearbyInnerStmtWM(s1, s2)
12         end
13     end
14     return R
15 S ← getAllLeafStatements(P')
16 while S ≠ ∅ do
17     S ← FixNearbyRound(S)
18 end
```

(1) (2) (3) (4)

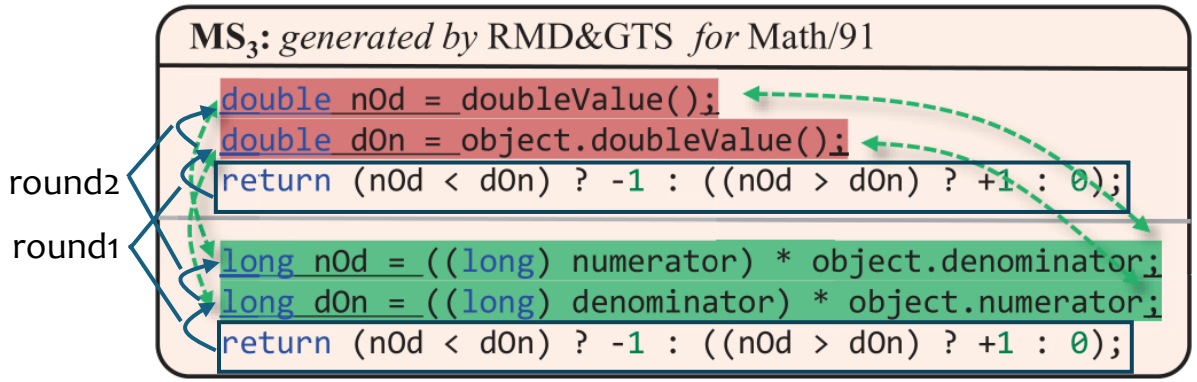
iteratively execute checks for newly mapped statements

Fixing Pass: The Last Four Passes

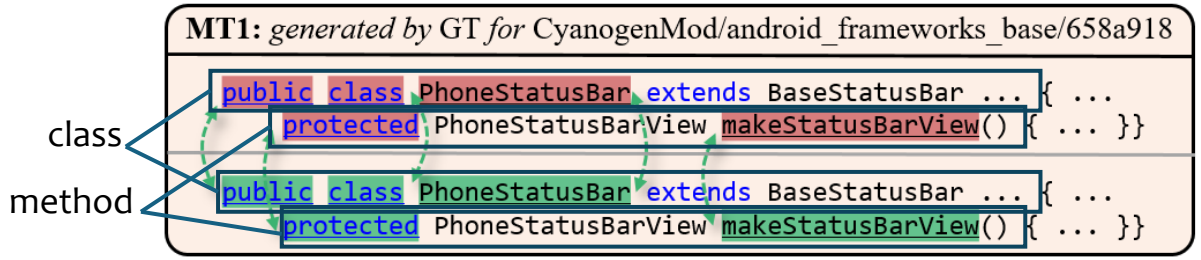
- **MatchInner** identifies **missing mappings within blocks**; uses an algorithm to find best candidates of method invocations
- **MatchParent** iteratively uses mapped nodes as anchors to identify their **missing-mapped parent nodes**



- **MatchNearby** iteratively identifies **missing-mapped nearby statements** with common elements



- **MatchChildren** identifies **missing-mapped children** by parent types and their roles



Evaluation

- **Research Questions**
  - **RQ1 (Effectiveness):** Can DIFFFIX effectively improve **node mapping accuracy**?
  - **RQ2 (Efficiency):** What is the **time cost** of DIFFFIX? Can optimizations improve its efficiency?
  - **RQ3 (Effect on ES Size)**: How does DIFFFIX affect **edit script size**?
  - **RQ4 (Impact of Subprocesses):** What's the contributions and costs of its **subprocesses**?

- **Subjects: 5 state-of-the-art ASTDiff techniques**
  - RMD, IAM, DAT, GTS, GT

- **Dataset**
  (1) DiffBenchmark: human-verified oracle diffs for 987 commits (for RQ1-4)
  (2) Manually crafted 10,607 diffs from commits in 116 popular GitHub projects (for RQ1)

Effectiveness (RQ1)

- **Results on DiffBenchmark** (800 Defects4j commits)
  - *Metrics*: PDR (Perfect Diff Rate) *perfect diff*: a diff without inaccurate mappings
  - *Summary*: (1) DIFFFIX **improved PDR by 5.25%-51.12%** on the ASTDiff techniques
    (2) DIFFFIX applied to RMD achieved **the best PDR of 95.38%**
- **Results on 10,607 diffs** (generated by RMD)
  - DIFFFIX **modified 1,688 (15.91%) diffs** from commits in 115 projects
  - Randomly selected 320 of the 1,688 diffs to manually verify fixing accuracy
  - *Summary*: DIFFFIX accurately fixed 298 of the 320 diffs, **achieved a precision of 93.12%**

	PDR_{stmt} (%)	PDR_{token} (%)	Missing Mappings (#)	Arbitrary Mappings (#)	Wrong Mappings (#)
RMD	89.38	86.00	719	81	251
RMD+DIFFFIX	97.62 (+8.25)	95.38 (+9.38)	164 (–555)	81 (+0)	42 (–209)
IAM	90.38	71.00	570	384	408
IAM+DIFFFIX	92.25 (+1.88)	80.75 (+9.75)	353 (–217)	400 (+16)	372 (–36)
DAT	82.00	70.62	177	1,279	986
DAT+DIFFFIX	86.50 (+4.50)	75.88 (+5.25)	174 (–3)	1,162 (–117)	787 (–199)
GTS	72.25	62.62	1,424	356	1,006
GTS+DIFFFIX	82.88 (+10.62)	72.25 (+9.62)	759 (–665)	362 (+6)	801 (–205)
GT	75.25	17.25	3,374	717	1,035
GT+DIFFFIX	82.88 (+7.62)	68.38 (+51.12)	892 (–2,482)	637 (–80)	831 (–204)

(2) (1)

PDR_{stmt}: only stmt mappings PDR_{token}: check all mappings

Answer to RQ1:

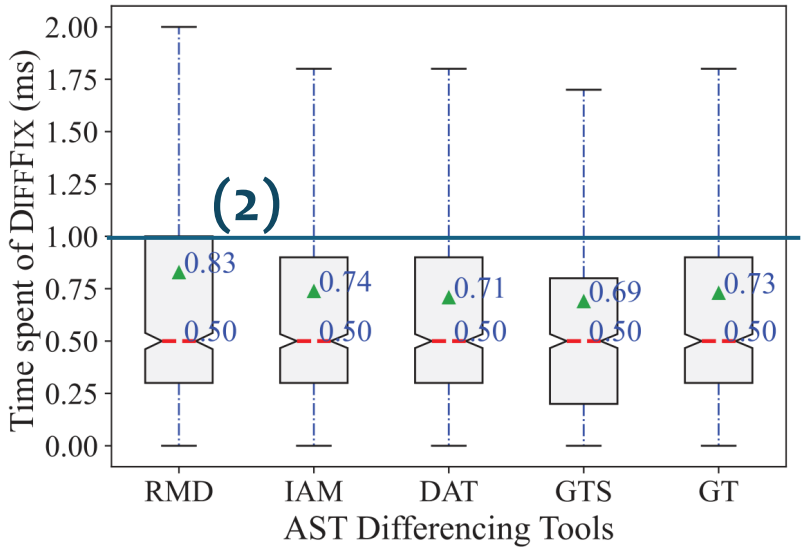
DIFFFIX effectively improved the accuracy of diffs generated by five techniques for real-world commits, demonstrating high precision and generalizability.

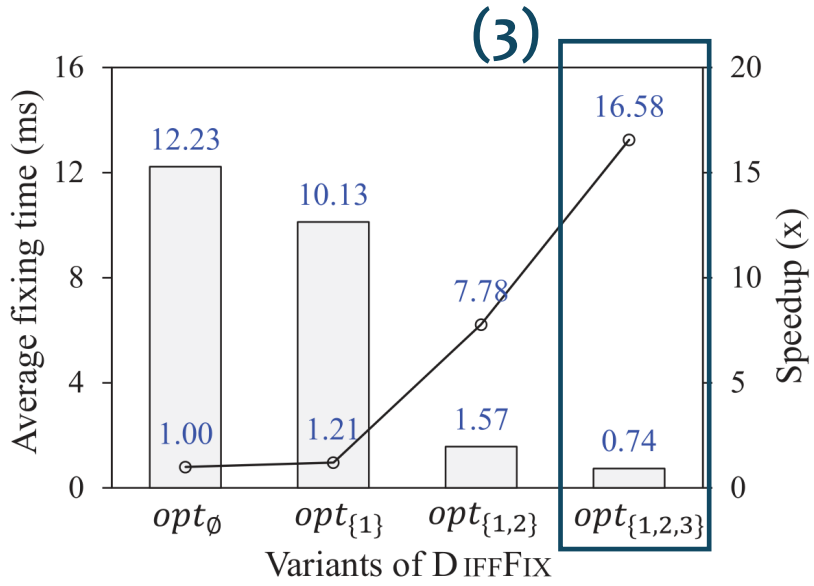
Efficiency (RQ2)

(1) DIFFFIX has **a negligible overhead (<2%)** on four techniques, and take ≈1/4 of GTS matching time
(2) DIFFFIX processed most diffs in **less than 1 *ms***, the maximum is 9.4-13.0 *ms* across techniques
(3) Three optimizations improved DIFFFIX's efficiency, ultimately achieving a **16.58x speedup**

(1)

	T_{diff} (ms)
RMD	34,108.1
RMD+DIFFFIX	34,769.5 (↑1.94%)
IAM	96,568.5
IAM+DIFFFIX	97,158.0 (↑0.61%)
DAT	32,472.9
DAT+DIFFFIX	33,038.4 (↑1.74%)
GTS	2,206.3
GTS+DIFFFIX	2,757.1 (↑24.96%)
GT	370,413.4
GT+DIFFFIX	370,995.9 (↑0.16%)





Answer to RQ2:

DIFFFIX demonstrated high efficiency with negligible time cost on most techniques. The optimizations significantly improved its efficiency.

Effect on Edit Script Size (RQ3)

(1) DIFFFIX reduced ES size on four techniques (↓3~11 actions per diff on average)
(2) DIFFFIX slightly increased ES size on DAT (↑3 actions per diff on average)

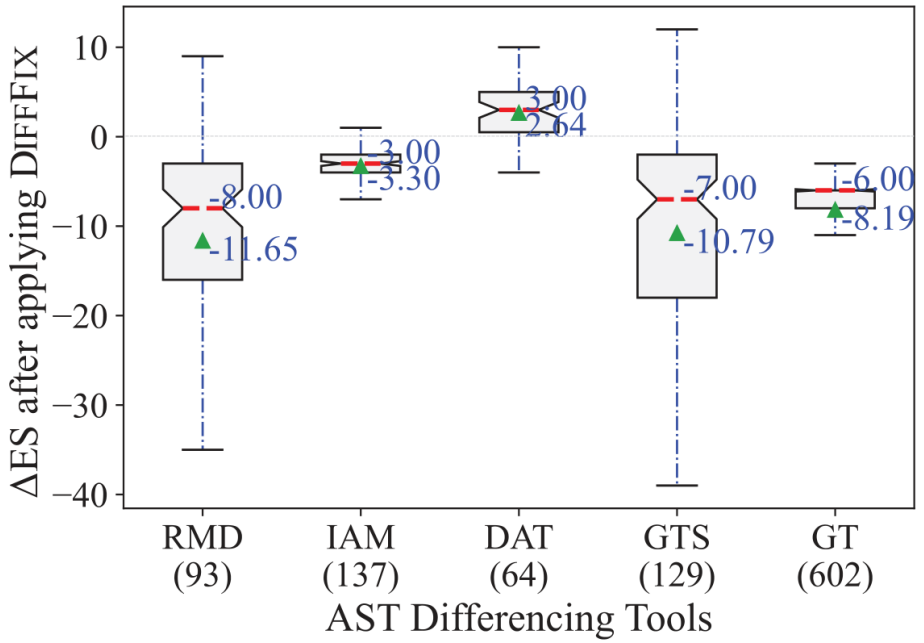
fixing missing mappings

↓

reduce unnecessary deletions&additions

↓

↓ES Size



fixing arbitrary mappings

↓

cancel confusing moves&updates

↓

↑ES Size

Answer to RQ3:

DIFFFIX reduced ES size on most techniques by fixing inaccurate mappings.

Impact of Subprocesses (RQ4)

(1) The warm-up phase fixed some inaccurate mappings in RMD and IAM related to identical subtrees

(2) The contributions of fixing passes were widely distributed across techniques

(3) The warm-up phase took up longer time than the whole fixing phase

(4) All passes had execution times of the same order of magnitude, and MatchChildren (MC) took the most time.

(a) The number of diffs that subprocesses take effect on (#)

	Warm-up Phase (1)	Fixing Passes (2)								
		MU	FP	FC	FI	FN	MI	MN	MP	MC
RMD	8	**45**	4	7	7	11	22	16	20	12
IAM	**30**	4	7	7	3	20	**38**	1	26	75
DAT	0	4	**10**	56	**33**	23	1	2	4	10
GTS	0	41	1	**71**	4	26	25	**30**	**29**	7
GT	0	26	4	70	23	**27**	3	12	20	**681**

(b) Total time spent of subprocesses (ms)

	Warm-up Phase (3)	Fixing Passes								(4)
		MU	FP	FC	FI	FN	MI	MN	MP	MC
RMD	377.6	13.5	7.9	13.6	15.5	16.8	34.9	10.1	18.1	53.1
IAM	**386.8**	13.9	**9.9**	16.2	18.4	19.0	**37.0**	12.7	24.7	58.4
DAT	371.7	8.5	**9.9**	14.9	**20.3**	**22.5**	34.0	12.3	**25.0**	**62.8**
GTS	365.9	13.7	7.7	15.6	18.1	20.0	33.9	13.5	19.5	60.1
GT	384.4	**17.3**	9.0	**16.3**	18.0	20.1	32.6	**14.5**	23.4	62.4

Answer to RQ4:

All fixing passes have contributions that vary across techniques.

The warm-up phase bore the main time cost among subprocesses.

Discussion

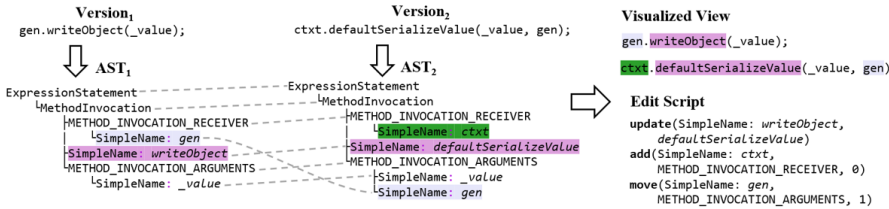
- **Key Features of DIFFFIX**
  - An **effective** and **efficient** AST diff fixing approach
  - Post-processing, **incremental**, **context**(↓FN)+**type**(↓FP), optimized
- **Main Implications**
  - Inaccurate mappings exhibit certain **frequent and generalizable patterns**
  - Context and type information guide establishing accurate node mappings
  - Coexisting inaccurate mappings can be incrementally resolved
- **Future Work**
  - Tackle **subtle inaccurate mappings**
    - mainly related to variables/method invocations
    - introduce lightweight **semantic analysis** into DiffFix
  - Establish a continuously updated public **AST diff database**
    - report inaccurate diffs / provide verified accurate diffs
  - Investigate **the impact of ASTDiff accuracy on downstream tasks**
    - conduct systematic studies

Summary

Overview of AST Differencing (ASTDiff)

- **Process**
  (1) parse two versions of programs into ASTs
  (2) compare ASTs to get the diff (node mappings)
  (3) generate edit script / visualize syntactic change

A fundamental approach to capture structural and syntactic changes of program elements



- **Techniques**
  ChangeDistiller, GumTree, MTDiff, IJM, iASTMapper, RM-ASTDiff, etc.
- **Applications**
  automated program repair, change pattern mining, refactoring detection, etc.

Accuracy Issue of ASTDiff Techniques

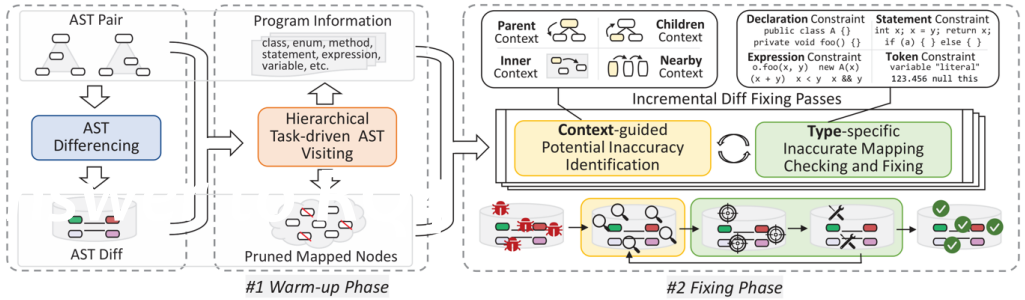
- **Can the generated diffs accurately reflect the code changes?**
- **Limitations of existing ASTDiff techniques**
  - Unsound node matching heuristic
  - Insufficient use of program information
  - Trade-off between accuracy and efficiency
  - Mutual interference of inaccuracies

  } Struggle to generate accurate diffs, resulting in **20%-40% inaccuracies**[1,2]

Motivation:
Since inaccurate mappings are difficult to prevent, can we identify and fix them in the generated diff to improve the accuracy?

[1] Y. Fan, X. Xia, D. Lo, et al. A differential testing approach for evaluating abstract syntax tree mapping algorithms. ICSE, 2021.
[2] P. Alikhanifard and N. Tsantalis. A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools. TOSEM, 2025.

Our Approach: DIFFFIX



- **Input:** A pair of ASTs and their diff generated by an ASTDiff technique
- **Output:** A refined diff with inaccurate mappings fixed

#1 Warm-up Phase performs initializations and optimizations
#2 Fixing Phase performs identification and fixing of inaccuracies

Discussion

- **Key Features of DIFFFIX**
  - An **effective** and **efficient** AST diff fixing approach
  - Post-processing, **incremental**, **context**(↓FN)+**type**(↓FP), optimized
- **Main Implications**
  - Inaccurate mappings exhibit certain **frequent and generalizable patterns**
  - Context and type information guide establishing accurate node mappings
  - Coexisting inaccurate mappings can be incrementally resolved
- **Future Work**
  - Tackle **subtle inaccurate mappings**
    - mainly related to variables/method invocations
    - introduce lightweight **semantic analysis** into DiffFix
  - Establish a continuously updated public **AST diff database**
    - report inaccurate diffs / provide verified accurate diffs
  - Investigate **the impact of ASTDiff accuracy on downstream tasks**
    - conduct systematic studies

The 40th IEEE/ACM International Conference on Automated Software Engineering (ASE 2025)

北京科技大学 University of Science and Technology Beijing

SWINBURNE UNIVERSITY OF TECHNOLOGY

Thank You!

Paper

Code

Guofeng Zeng
d202110394@xs.ustb.edu.cn

Chang-ai Sun
casun@ustb.edu.cn

Kai Gao
kai.gao@ustb.edu.cn

Huai Liu
hliu@swin.edu.au

Seoul, November 19, 2025